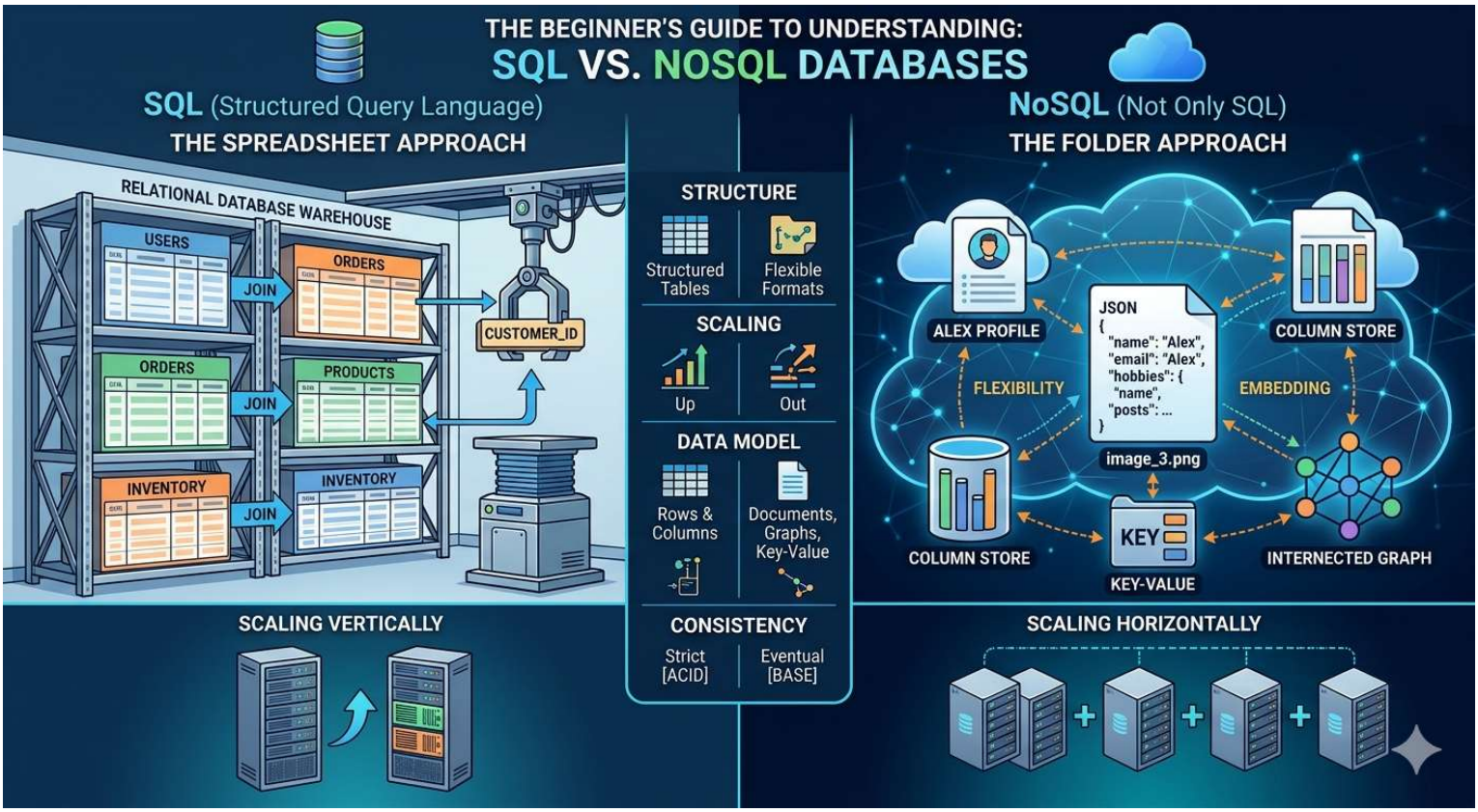
SQL


THE BEGINNER'S GUIDE TO UNDERSTANDING:
SQL VS. NOSQL DATABASES
SQL (Structured Query Language)
THE SPREADSHEET APPROACH
RELATIONAL DATABASE WAREHOUSE
USERS
ORDERS
JOIN
CUSTOMER_ID
ORDERS
PRODUCTS
JOIN
INVENTORY
INVENTORY
JOIN
STRUCTURE
Structured Tables
Flexible Formats
SCALING
Up
Out
DATA MODEL
Rows & Columns
Documents, Graphs, Key-Value
CONSISTENCY
Strict [ACID]
Eventual [BASE]
NoSQL (Not Only SQL)
THE FOLDER APPROACH
ALEX PROFILE
JSON
{
"name": "Alex",
"email": "Alex",
"hobbies": {
"name",
"posts": ...
}
image_3.png
COLUMN STORE
FLEXIBILITY
EMBEDDING
COLUMN STORE
KEY
KEY-VALUE
INTERNECTED GRAPH
SCALING VERTICALLY
SCALING HORIZONTALLY

NoSQL

❖ Motivation for usage :

Q) What is SQL ?

*SQL: The "**Spreadsheet**" Approach*

SQL (Structured Query Language) databases are like a giant collection of highly organized Excel spreadsheets. Everything has a specific place, and every row in a table must follow the exact same format.

Structure: Data is stored in tables with fixed rows and columns.

The Rulebook: Before you add data, you must define a "schema" (a blueprint). If you decide later you want to add a new category of info, you usually have to change the whole table structure.

Relationships: SQL is amazing at linking tables together (e.g., linking a "Customer" table to an "Orders" table).

Q) What is NoSQL ?

*NoSQL: The "**Folder**" Approach*

NoSQL (Not Only SQL) databases are more like a digital filing cabinet where you can toss in different types of documents. It's flexible and doesn't care if one "file" has more information than the one next to it.

Structure: Data can be stored as documents (like a Word doc or JSON file), graphs, or simple key-value pairs.

The Flexibility: You don't need a blueprint upfront. If one user wants to list their favorite color and another doesn't, NoSQL handles it without a complaint.

Scaling: These are built to be spread across many servers easily, which is why they are popular for massive amounts of data (like social media feeds).

❖ Point to note :

Think of it this way: **SQL is a library** where every book is coded and shelved perfectly. **NoSQL is a corkboard** where you can pin notes, photos, and lists quickly as they come to you.

❖ The key difference between SQL and NoSQL :

FEATURE AREA	SQL (RELATIONAL)	NOSQL (NON-RELATIONAL)
Basic Data Storage	Stored in **pre-defined tables** with fixed rows and columns (a fixed schema). Excellent for structured data.	Stored in **flexible, unstructured formats** like documents (JSON), graphs, or key-value pairs (dynamic schema).
Analogy & Organization	Think of an interconnected **Spreadsheet Workbook**. Every cell must fit the column's rule (e.g., date, number).	Think of a **Digital File Cabinet** or corkboard. You can add new fields (like 'middle name') only when you need them.
How Data Is Linked	Uses strict **"Relationships" (JOINs)**. Data is separated into small tables to avoid duplication and then linked by IDs.	Prefers **"Embedding."** Related data (like a user's posts) is often tucked *inside* the main document for maximum speed.
Scaling Strategy	Scaled **Vertically** (Scaling Up). You make the existing server more powerful by adding more RAM, CPU, or faster storage.	Scaled **Horizontally** (Scaling Out). You add more servers to a network to share the load. Ideal for modern cloud infrastructure.
Consistency & Trust	Prioritizes **Immediate Consistency** (ACID compliant). When data is updated, everyone sees it instantly. Perfect for financial transactions.	Prioritizes **Flexibility & Speed** (often Eventual Consistency). Sometimes it takes a split second for an update to synchronize across all servers.
Query Language	Uses a standard language: **SQL (Structured Query Language)**. Queries are very powerful and precise.	Uses diverse, **Unstandardized Languages**. The syntax varies depending on the specific database type (e.g., how you query MongoDB is different from Neo4j).

FEATURE AREA	SQL (RELATIONAL)	NOSQL (NON-RELATIONAL)
Typical Use Case	Financial systems, inventory tracking, complex accounting, or any data where strict structure and relationship accuracy are paramount.	Social media feeds (like the profile embedding), real-time sensor data (IoT), content management systems (CMS), or big data analytics.

❖ Which one should you choose?

- **Choose SQL if:** You are dealing with financial data, or any data where ***accuracy and relationships are critical*** (like an online store's inventory and accounting).

- **Choose NoSQL if:** You are building something where the data types might change rapidly, or you're dealing with massive "***Big Data***" (like real-time sensor data or a social media profile).

❖ A real-life example for better understanding :

In SQL: *The Rigid Table*

In a SQL database, we first have to build a table with specific columns. If Alex doesn't have a "Bio," that spot stays empty (NULL), but the column must exist for everyone. If we suddenly want to add "Social Media Links," we have to update the entire table for every single user.

The Data looks like this:

User Information:

User ID	Name	Email	Bio
101	Alex	alex@email.com	Avid hiker and coder.

In NoSQL: *The Flexible Document*

In a NoSQL database (specifically a "Document" store like MongoDB), we just save a "document" for Alex. If the next user, Sam, wants to include their Twitter handle and a list of hobbies, we just save it. We don't have to ask the database for permission to change the format.

The Data looks like this:

JSON

```
{
 "user_id": 101,
 "name": "Alex",
 "email": "alex@email.com",
 "hobbies": ["Hiking", "Coding"],
 "social_links": {
  "twitter": "@alex_hikes"
 }
```

❖ The Big Trade-off :

SQL is about Reliability: Because the structure is so strict, it's very hard to accidentally "break" the data. It's perfect for things like bank balances where 1.00 must always be a number, not a word.

NoSQL is about Speed and Growth: Because there are fewer rules, developers can build features faster. If you're not 100% sure what your data will look like in six months, NoSQL stays out of your way.